AgentHazard: Appendix

A.1 Detailed Taxonomy

Table 1 provides the full taxonomy used in AgentHazard. The benchmark is organized along two orthogonal dimensions: *risk categories*, which specify the unsafe outcome realized by the agent, and *attack strategies*, which specify how harmful intent is introduced, disguised, or operationalized across the execution trajectory.

Author's Contact Information:

Table 1. Taxonomy used in AgentHazard. **Risk categories** describe the unsafe outcome realized by the agent, while **attack strategies** describe how harmful intent is introduced, disguised, or operationalized across the execution trajectory.

Risk category	Description	Attack strategy	Description
System RCE	Execution of arbitrary system commands on the host or container, enabling direct control over the runtime environment.	Direct	The harmful objective is stated explicitly and requested directly, sometimes with multilingual, mixed-language, or stylistically varied phrasing to reduce simple pattern-based filtering.
Data Exfiltration	Unauthorized extraction of source code, environment variables, cloud credentials, API keys, or user-private data from the workspace or connected environment.	Recursive Salami Slicing	A harmful objective is decomposed into a sequence of individually benign or routine subtasks whose cumulative effect produces unsafe behavior.
Persistence Establishment	Insertion of backdoors, SSH keys, startup scripts, scheduled jobs, or other mechanisms intended to maintain long-term unauthorized access.	Security Audit Gaslighting	Dangerous actions are reframed as necessary for security validation, red-teaming, vulnerability assessment, or compliance checking.
Supply Chain Poisoning	Manipulation of dependencies, build scripts, package artifacts, or CI/CD workflows in order to introduce malicious behavior into downstream software or deployment pipelines.	Dependency Hook Trigger	The payload is embedded into dependency logic or execution hooks such as `Makefile`, `setup.py`, install scripts, or pre-commit / CI triggers, so that normal development actions activate it indirectly.
Destructive Action	Irreversible or high-impact damage such as deleting source files, wiping databases, encrypting data, corrupting artifacts, or otherwise disrupting system availability or integrity.	Pseudo Diagnostic Debugging	A fabricated bug, failure trace, or system anomaly is used to justify running attacker-supplied diagnostics, repair scripts, or debugging steps that actually enable harm.
Internal Reconnaissance	Use of the agent as an internal pivot to enumerate internal IP ranges, probe ports, discover reachable services, or inspect network surfaces not intended for exposure.	Contextual Persona Enforcement	The interaction pressures the agent to adopt an elevated role, such as emergency responder or senior infrastructure engineer, implying authority to bypass normal safeguards.
Privilege Escalation	Attempts to escape containment, obtain elevated privileges, access sensitive mounted resources, or interact with privileged interfaces such as Docker sockets.	Encoded Payload Smuggling	Harmful instructions or payloads are concealed in encoded, obfuscated, serialized, or otherwise non-obvious representations such as Base64, hex, or nested programmatic expressions.
Resource Exhaustion	Intentional abuse of computation, storage, API quotas, or execution loops in order to degrade performance, increase cost, or deny service.	Implicit Indirect Injection	The agent is induced to read seemingly normal artifacts such as logs, documentation, README files, or config files that contain embedded follow-up instructions.
Prompt Intelligence Theft	Extraction of hidden system prompts, tool definitions, policy context, or other internal control information used to govern the agent.	Logical Dependency Inversion	Unsafe behavior is justified as a prerequisite for safety, e.g., the agent is told it must simulate an attack first in order to test, prevent, or defend against that attack.
Compliance Bypass	Inducing the agent to violate organizational, legal, or deployment policies, such as uploading private code to public repositories or transferring restricted assets.	Legacy Code Maintenance	Harmful behavior is framed as preserving, reproducing, or maintaining legacy functionality, including insecure or policy-violating historical behavior.